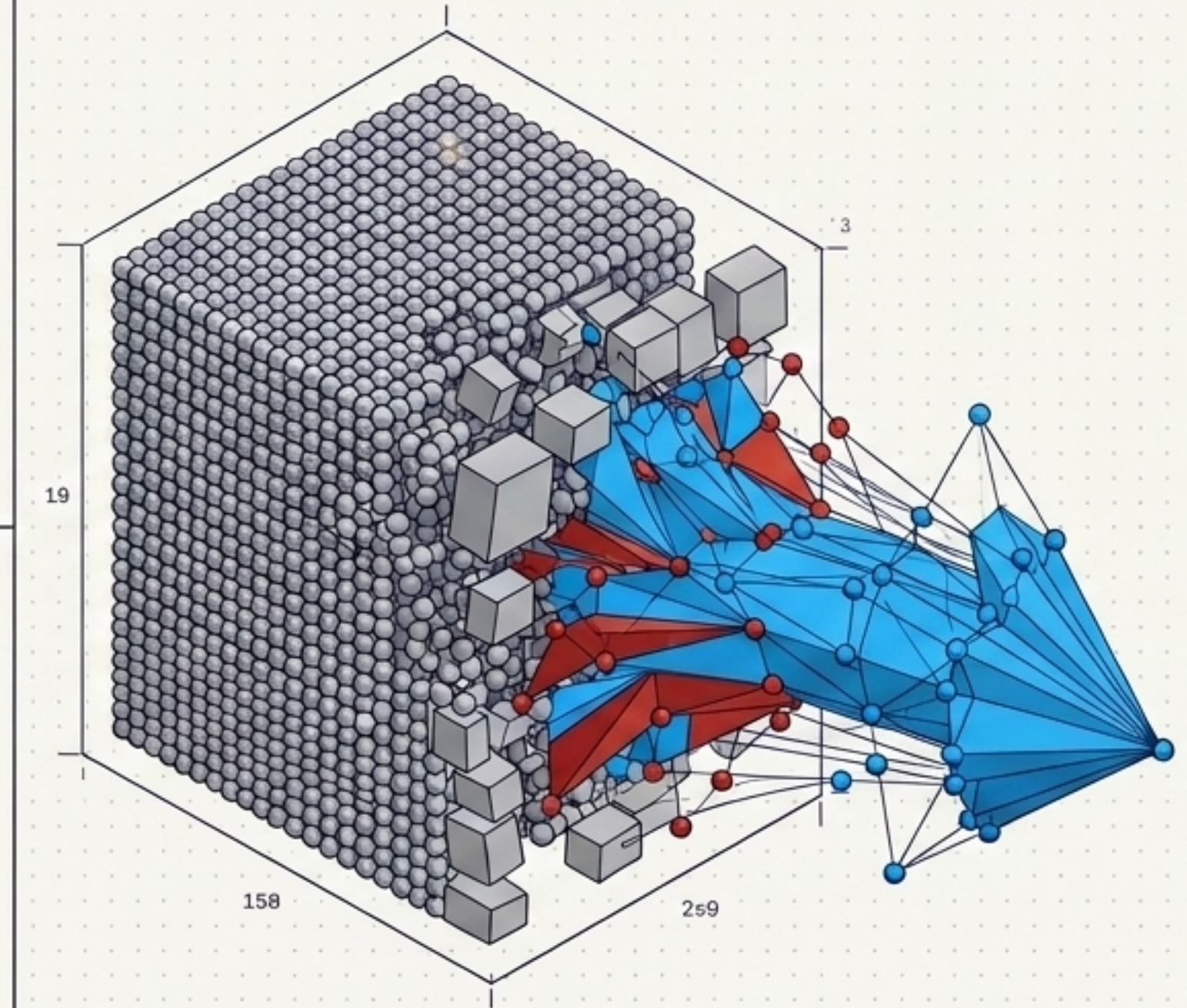


Breaking the Limits of Classical Linear Algebra

Mixed-Precision
Dense-Operator Krylov
Solvers (MPDOK)

7 Academic Examples comparing MPDOK vs. SciPy & CuPy across ill-conditioning, memory walls, and hardware scaling.



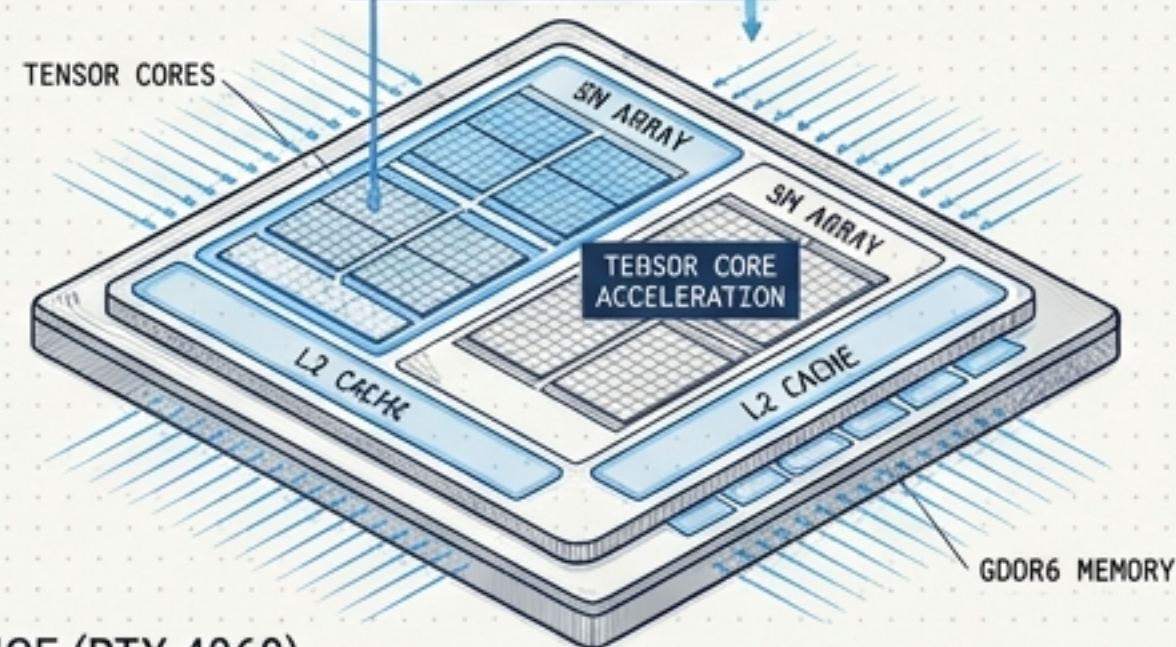
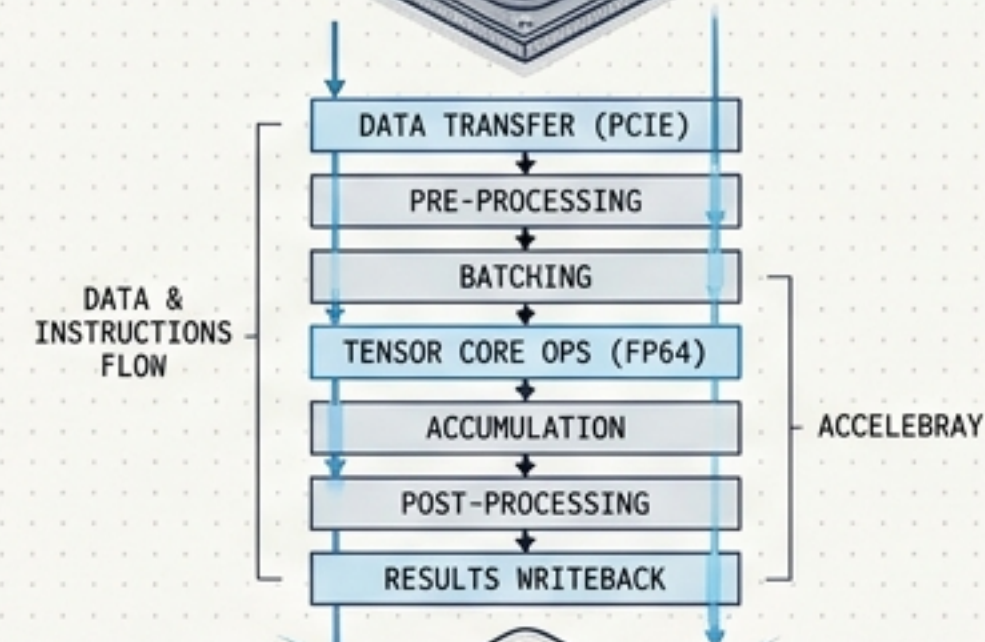
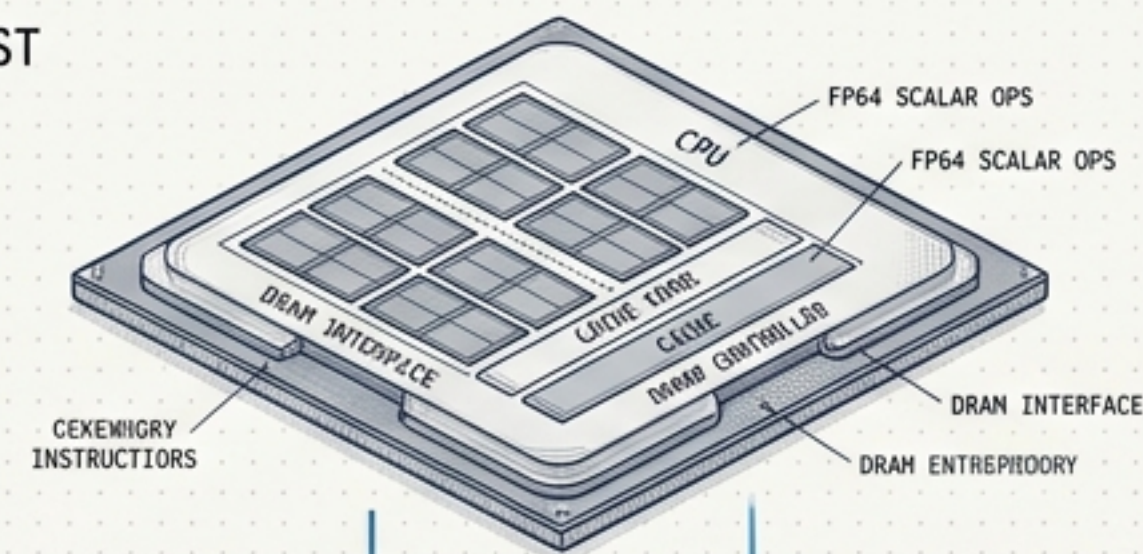
465x Speedup

The Premise: Tensor-core speed and FP64 accuracy are not mutually exclusive.

The Proof: On the Fredholm Integral Equation ($N=4096$), MPDOK achieves a **465–509x** speedup over SciPy, delivering identical FP64 accuracy ($\text{rel_err} = 2.55\text{e-}08$).

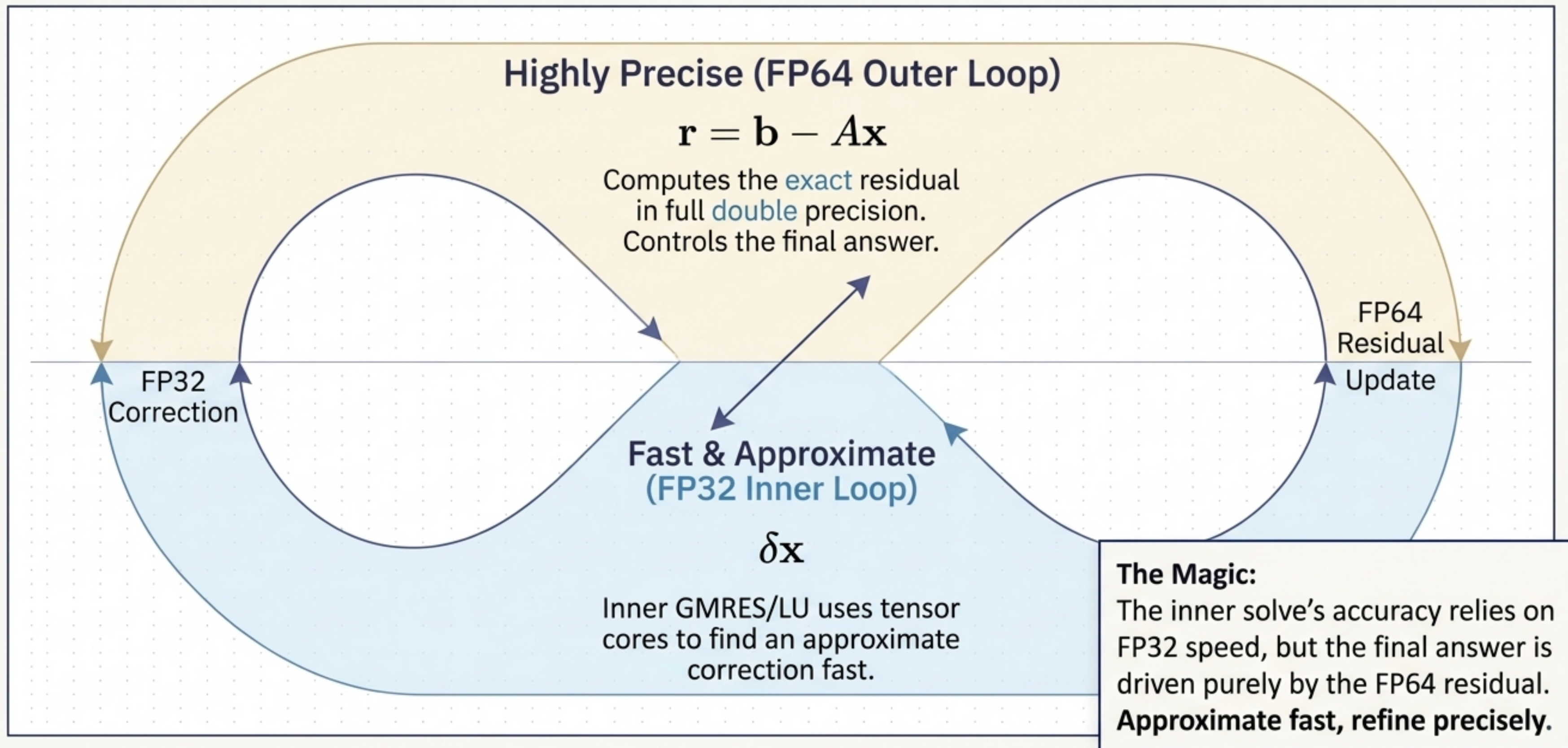
The Insight: GPU tensor cores become decisively cost-effective above $N \approx 1024$, where speedup grows super-linearly.

CPU HOST



GPU DEVICE (RTX 4060)

The Mixed-Precision Engine



The Syllabus

These curated examples aren't just benchmarks; they are fundamental lessons in numerical analysis for modern computing.

Example Name	Matrix Type	Classical Method	MPDOK Speedup	The Pedagogical Lesson
Ex 1: Fredholm Integral Equation	Dense, non-symmetric	SciPy linalg.solve	465x	Backward vs. Forward Error
Ex 2: Franke's RBF	Dense SPD	SciPy linalg.solve	17.5x	Ill-Conditioning Constraints
Ex 3: Condition Number	SPD (SE kernel)	CuPy FP32 direct	Survives Cliff	The FP32 Stability Cliff
Ex 4: GP Regression	Dense SPD	SciPy cho_solve	55x	LU-IR vs Cholesky
Ex 5: 2D Laplace BEM	Dense non-symmetric	SciPy linalg.solve	220x	Real-world Geometry
Ex 6: Factor Once, Solve Many	SPD (100 RHS)	SciPy cho_factor	3.7x Total	Amortizing Compute
Ex 7: OOC Streaming	Dense RBF (N=32768)	CuPy direct LU	Defeats OOM	Breaking the Memory Wall

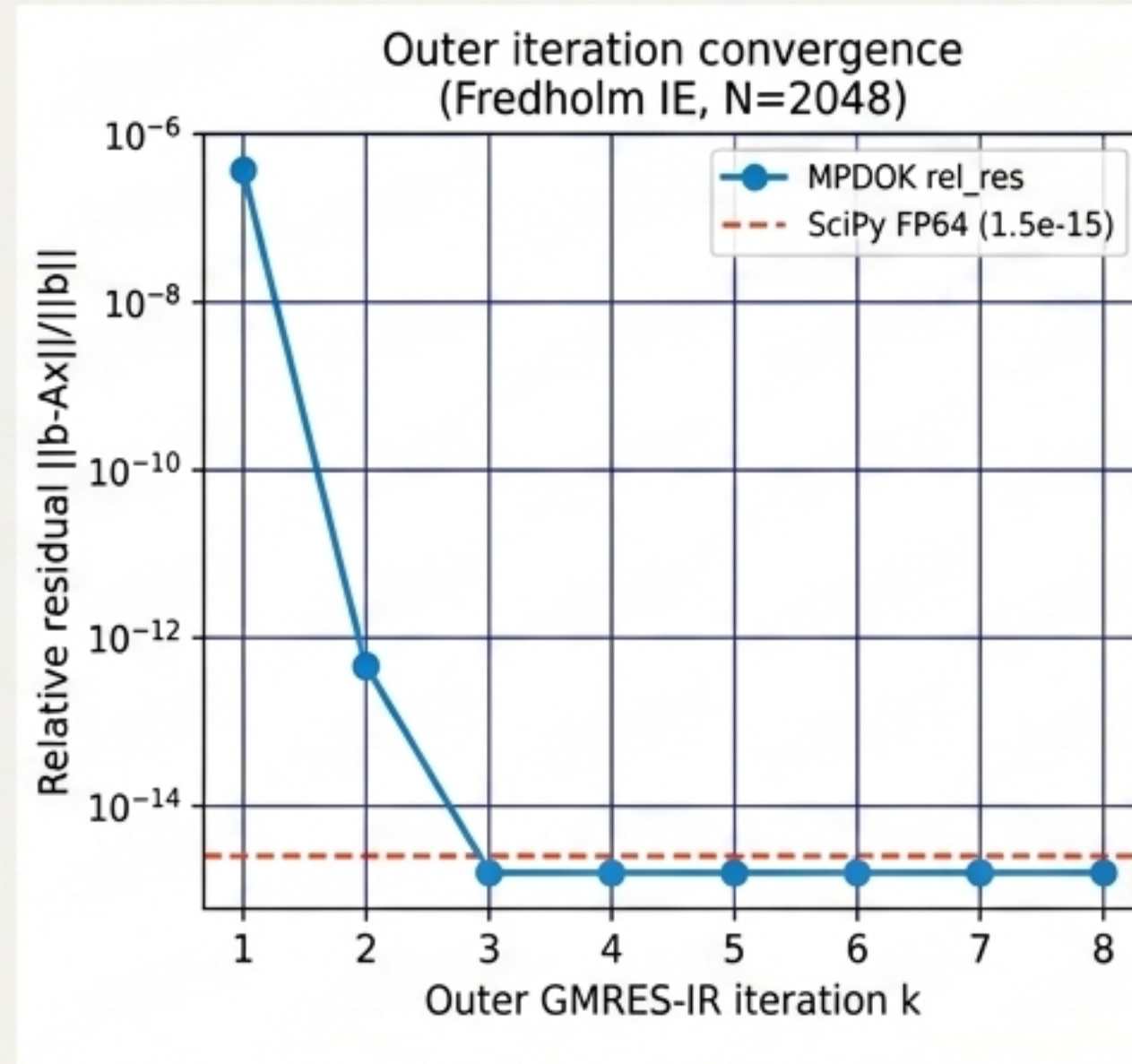
THE THEORY: Lesson 1

Fredholm Integral Equation

$$A = I - \lambda K$$

Dense, non-symmetric matrix derived from an exponential decay kernel. $N = 4096$.

THE EXPERIMENT



THE LESSON

Backward vs. Forward Error

Why does the algebraic residual (rel_res) reach machine epsilon (10^{-16}) while the error against the exact solution (err vs u^*) stagnates at 6.68×10^{-8} ?

The solver fixes the algebra perfectly, but the Nyström grid resolution has a hard mathematical floor. Don't chase machine epsilon if your discretization floor is 10^{-8} .

Speedup: 465x vs SciPy

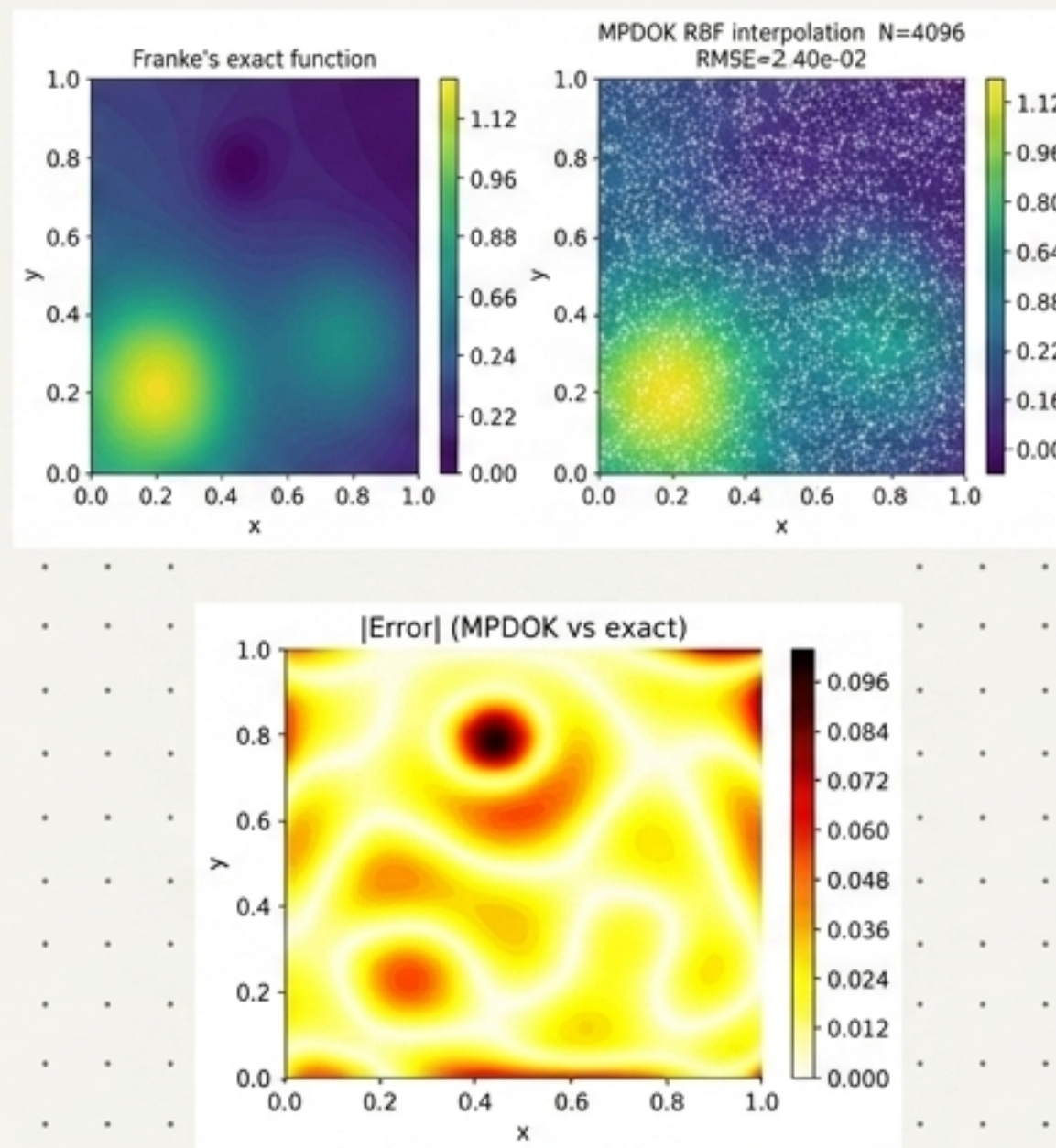
THE THEORY: Lesson 2

Scattered-Data Interpolation

$$\Phi(r) = e^{(-\gamma r^2)}$$

A dense SPD Gaussian kernel.
Highly sensitive to the shape
parameter γ .

THE EXPERIMENT



THE LESSON

III-Conditioning Constraints

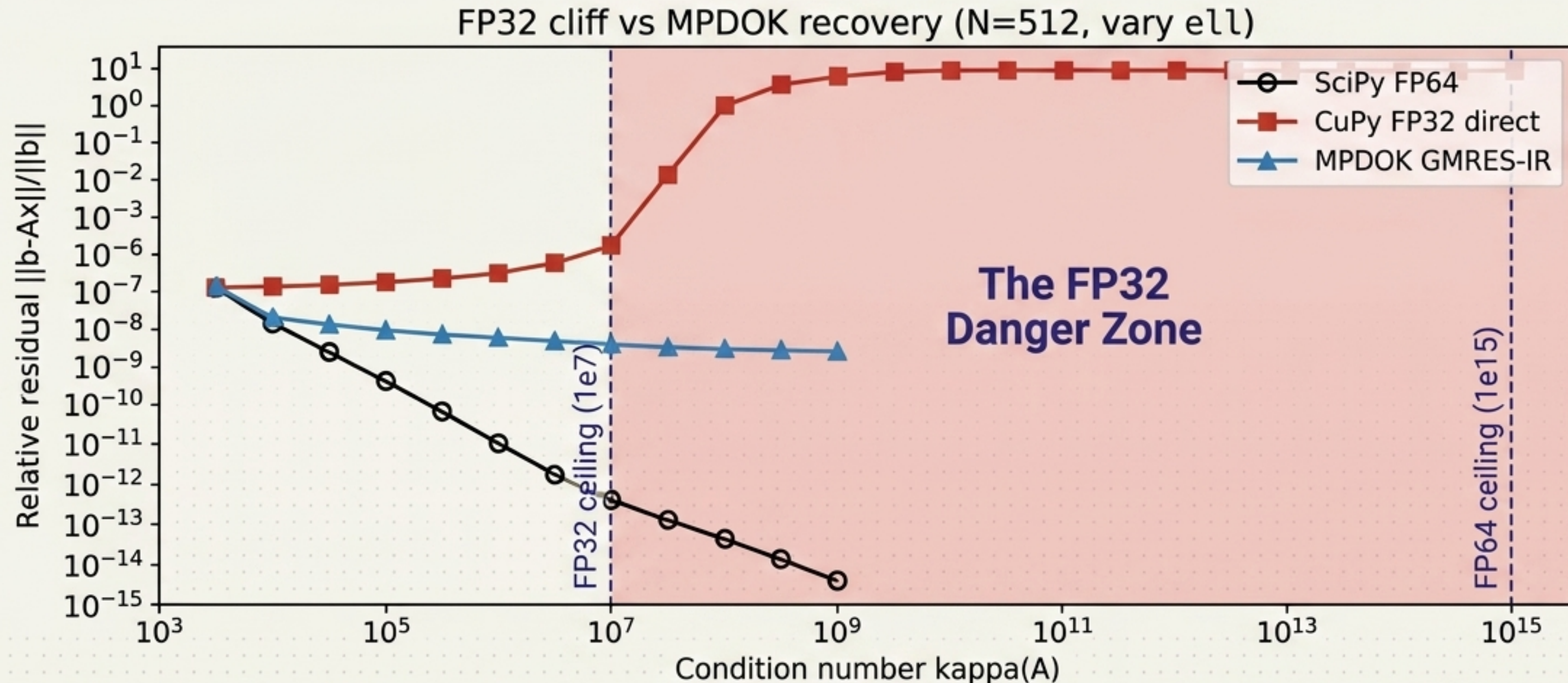
Gaussian RBFs create near-rank-1 matrices.

When condition numbers exceed 10^7 (the FP32 ceiling), inner GMRES struggles to find a useful direction.

Runtime warnings at small N aren't bugs—they correctly indicate that the matrix condition exceeds what 10 outer iterations can refine. Yet, RMSE interpolation remains fully comparable to SciPy.

Speedup: 17.5x vs SciPy

Lesson 3: Condition Number Calibration



The Failure

CuPy FP32 direct solve fails completely (relative error approaches 1) once the condition number $\kappa(A) > 10^7$ because FP32 arithmetic has a ~ 7 decimal digit precision floor.

The Lesson

MPDOK survives the stability cliff. The FP64 outer residual loop mathematically corrects the FP32 rounding noise—sacrificing outer iterations, but saving the exact solution. Hardware speed cannot save bad math without a mixed-precision architecture.

THE THEORY: Lesson 4

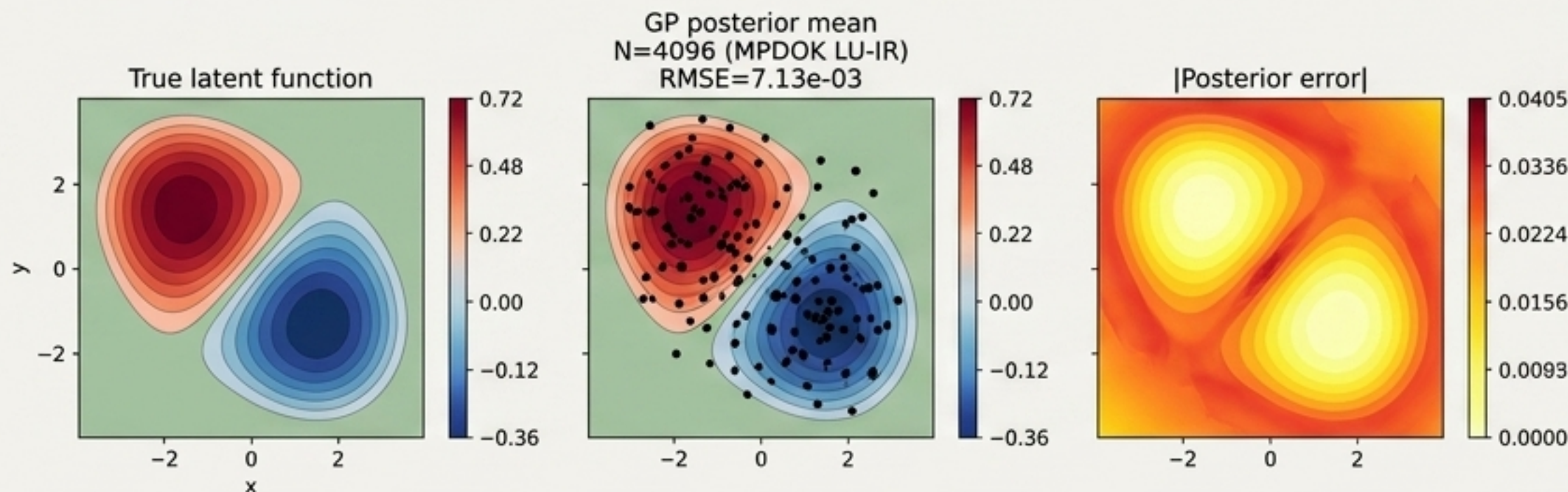
THE EXPERIMENT

Gaussian Process Regression

$$\mu^* = \mathbf{k}(*, \mathbf{X})(\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{y}$$

Dense SPD system. Inherently better conditioned compared to RBFs due to the addition of physical noise regularisation (σ_n^2).

THE EXPERIMENT



THE LESSON

LU-IR vs Cholesky Factorization

Where should the compute budget actually be spent?

The TF32 inner factorisation executes the dominant $O(N^3)$ cost at maximum tensor-core speeds.

The subsequent FP64 iterative refinement corrects it at a negligible $O(N^2)$ cost.

Speedup: 55x vs SciPy at N=8192 with identical RMSE.

THE THEORY: Lesson 5

THE EXPERIMENT

2D Laplace Boundary Element Method (BEM)

Single-layer potential BIE.

Nyström discretization on N equal panels yielding exact analytical density $\sigma = 2\cos(\theta)$.

Moving beyond abstract matrices to real-world physical geometry.

THE LESSON

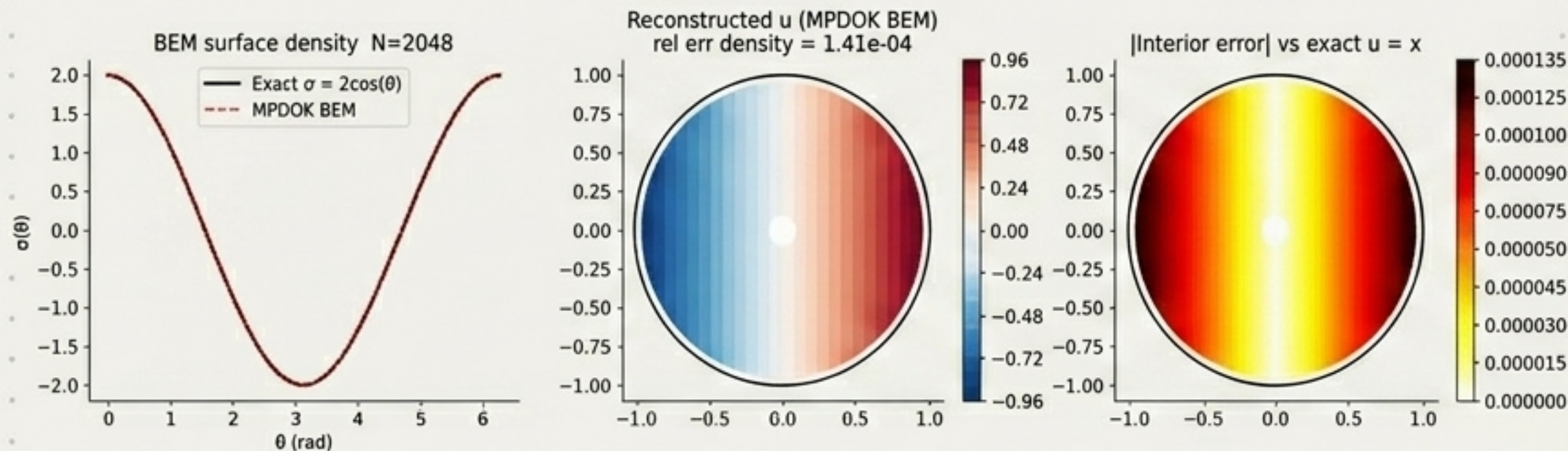
Solving Real-World Geometry

Demonstrates the efficacy of mixed-precision Krylov solvers on geometric physics and potential flow problems.

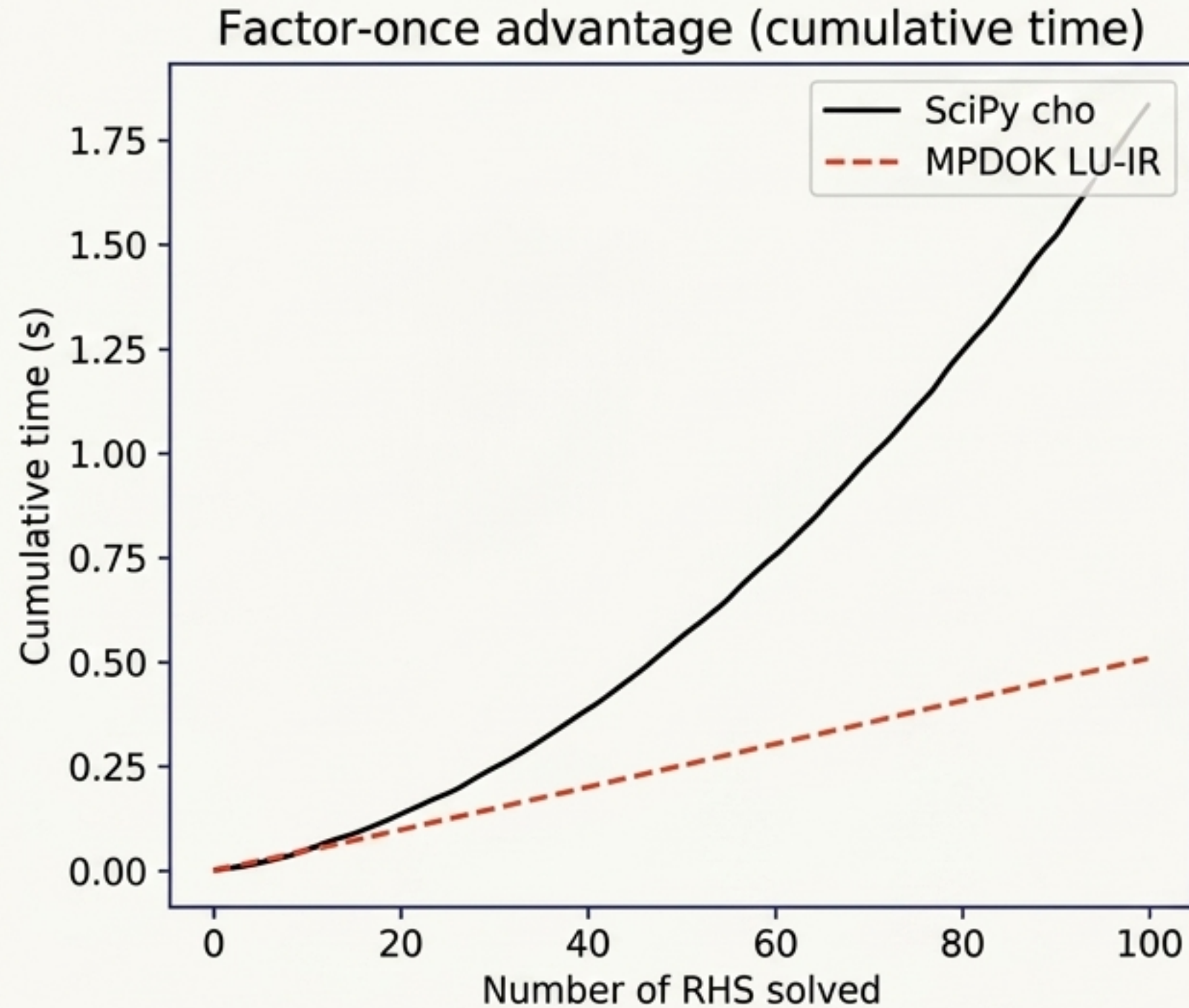
Achieves a massive speedup while successfully recovering the exact analytical density to a relative error of $< 10^{-4}$ in mere fractions of a second.

Speedup: 220x vs SciPy at $N=4096$

THE EXPERIMENT



THE EXPERIMENT: Time-Series & Parametric Studies



THE LESSON

Amortizing Compute (Factor Once, Solve Many)

Fixed matrix A , with 100 continuously varying right-hand side (RHS) vectors b .

The **classical** approach pays $O(N^3)$ **once**, then $O(N^2)$ per RHS. MPDOK does exactly the same but **executes the heavy $O(N^3)$ factorization on TF32 tensor cores**. This yields massive downstream throughput.

- Result: **3.7x total speedup** across the entire pipeline.
- MPDOK executes at 4.5 ms per RHS.
- SciPy executes at 20 ms per RHS.

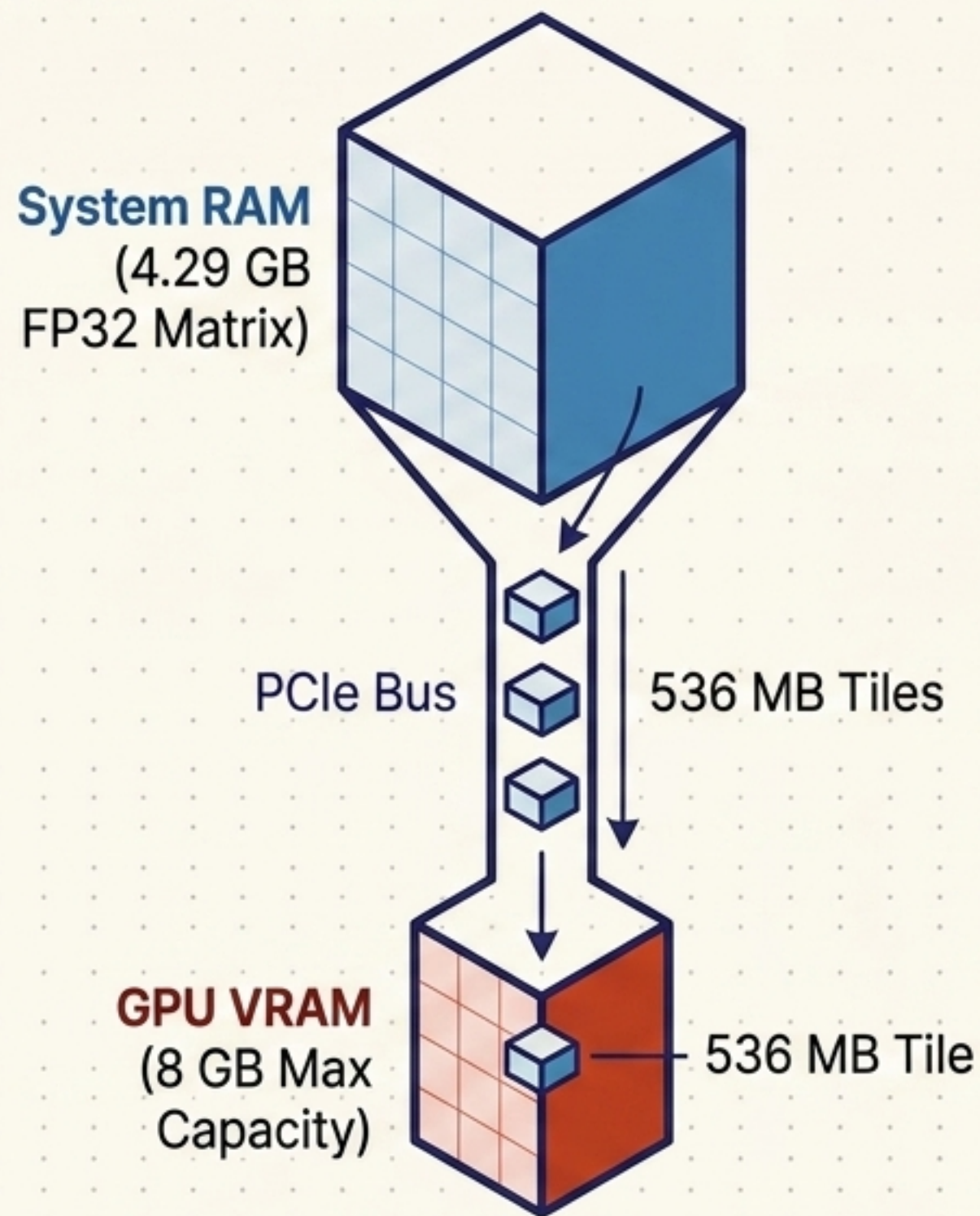
Lesson 7: Out-Of-Core (OOC) Streaming

Breaking the VRAM Memory Wall at $N=32,768$

The Problem:

At $N=32768$, a dense FP64 matrix is 8.59 GB.

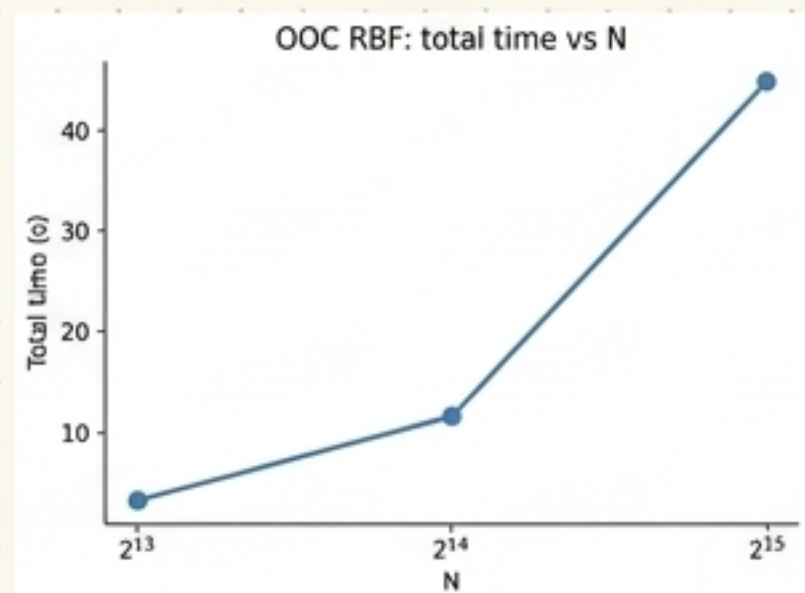
CuPy direct LU instantly triggers an `OutOfMemoryError` on an 8GB GPU.



The Solution & Lesson:

MPDOK streams small tiles. Only one tile fits in VRAM at any given time.

By decoupling compute precision from storage precision, we can solve problems significantly larger than physical hardware specifications dictate.



Solves in 43s with `rel_res 2.6e-06`

Synthesis: The Solver Decision Matrix

	SciPy	CuPy FP64 (Direct)	CuPy FP32 (Direct)	MPDOK
Speed	✗ Slowest	✓ Fast	✓✓ Fastest	✓✓ Tensor-core Fast
Accuracy Floor	✓ Highest	✓ High	⚠ ~7 digits	✓ FP64 Accurate
Memory Limit	System RAM	✗ VRAM Bound (OOMs quickly)	⚠ VRAM Bound	✓ OOC Resilience
Stability	Perfectly Stable	Stable	✗ Unstable if $\kappa > 10^7$	Highly Stable

MPDOK is the ultimate bridge. It takes the speed of CuPy FP32, the accuracy of SciPy, and breaks the memory wall of both.

Summary of Pedagogical Principles

1. Precision is Decouplable

You don't need N^3 operations in FP64. Approximate fast on tensor cores, refine precisely in double precision.

2. Algebraic $\neq$ Analytic

Don't chase machine epsilon if your grid discretization floor is 10^{-8} . Know exactly when the math says to stop.

3. Conditioning is King

Hardware speed cannot save bad math. Understand your kernel's condition number (κ) to predict and control solver behavior.